

JVM GC Log Analyzer

User Guide

A practical guide to analysing JVM garbage collection logs, interpreting performance indicators, reviewing throughput and heap memory screenshots, and exporting share-ready PDF reports.

Generated: 2026-05-20

Table of Contents

1. Overview
2. Benefits
3. How to Use the Application
 - 3.1 Starting the application
 - 3.2 Uploading or selecting a GC log file
 - 3.3 Running the analysis
 - 3.4 Reviewing the results
 - 3.5 Interpreting graphs, metrics, and observations
 - 3.6 Downloading the analysis report as a PDF
 - 3.7 Using the Analyze Another File button
 - 3.8 Throughput and Heap Memory image analysis (optional)
4. Screenshots Gallery
5. Output Report
6. Supported Formats and Limits

1. Overview

The **JVM GC Log Analyzer** is a Python-based tool that helps engineers and support staff understand how a Java Virtual Machine is managing memory. It parses garbage-collection (GC) log files produced by the JVM and turns them into clear visualisations, metrics, and plain-language observations.

Garbage collection is the JVM's automatic memory-management process: it reclaims memory no longer in use so the application can continue allocating new objects. When GC behaviour goes wrong — long pauses, frequent Full GCs, climbing heap usage — applications slow down, jitter, or run out of memory entirely. GC logs record everything the GC does, but they are dense, repetitive, and difficult to read by eye.

The analyzer removes that friction. Upload a log file and the tool surfaces what matters: pause-time distribution, throughput percentage, heap-reclaim trends, GC-type breakdown, and rule-based observations tagged with severity. A downloadable PDF report bundles everything for sharing with teammates, attaching to incident tickets, or archiving alongside release notes.

The tool runs locally as a web application on *http://localhost:5000* — no data leaves your machine, which makes it safe to use on production logs that may contain internal hostnames, timestamps, or other sensitive context.

Supported GC log formats

- **JDK 8** — `-XX:+PrintGCDetails` output
- **JDK 9+** — Unified logging via `-Xlog:gc*`
- **G1GC**, **ZGC**, and **Shenandoah** collector formats
- Wrapper-format logs (Tanuki/jsvc-style with wall-clock timestamps)

2. Benefits

The analyzer is designed to reduce the time and expertise needed to make sense of a GC log. The benefits below describe the practical value it provides during day-to-day work, incident response, and performance tuning.

Faster GC log analysis

What previously took an experienced engineer fifteen to thirty minutes of grep-and-eyeball review is reduced to seconds. Upload a log, see the analysis.

Easier identification of memory and GC issues

Rising heap-before-GC trends, excessive Full GCs, p99 pause-time spikes, and unusual collector behaviour are surfaced as severity-tagged observations rather than buried in thousands of log lines.

Visual trend analysis

Heap-before-GC, heap-after-GC, GC duration over time, reclaimed memory over time, and pause-time distribution are all rendered as charts that make patterns immediately visible.

Better understanding of JVM performance

Computed KPIs — throughput percentage, CPU-time spent in GC, pause percentiles (p50/p95/p99), GC rate, total reclaimed memory — let you quantify how the JVM is performing rather than guessing.

Support for troubleshooting application health

When an application slows down, OOMs, or jitters, the GC log frequently holds the answer. The analyzer's observation engine turns the technical signal into actionable next steps.

Downloadable analysis reports

One click produces a professionally formatted PDF you can attach to a ticket, share in a PR review, or archive alongside a release. Reports include all charts, tables, observations, and a timestamp.

Extended image analysis

Beyond GC log parsing, the tool can also analyse throughput and heap-memory screenshots exported from the Management Console, comparing 30-day, 7-day, and 24-hour views to detect short-term issues that long-term graphs might hide.

No data leaves your machine

The application runs locally. Production logs and screenshots remain on your workstation — important for sites where uploading logs to a third-party cloud service is not allowed.

3. How to Use the Application

This section walks through the analyser end-to-end: starting the application, uploading a log, reviewing results, interpreting findings, and exporting the report.

3.1 — Starting the application

From a terminal in the project folder, install the dependencies (the first time only) and start the web interface:

```
pip install -r requirements_gc.txt

python gc_web.py
```

Open `http://localhost:5000` in a browser. The first screen you see is the upload page.

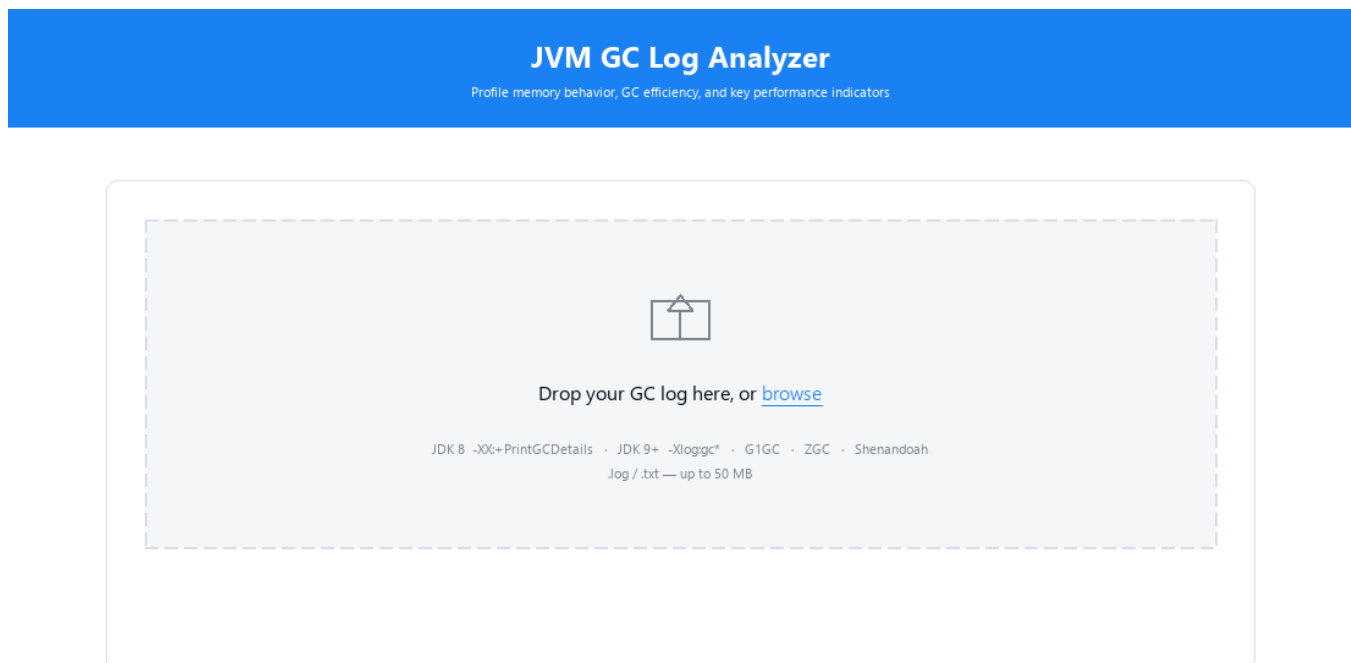


Figure 1 — The upload screen. Drag a GC log file onto the dashed area, or click 'browse' to pick one from disk.

3.2 — Uploading or selecting a GC log file

There are two ways to provide a log:

- **Drag-and-drop** — drag the log file from File Explorer (or Finder) onto the dashed drop area. The drop area highlights when a file hovers over it.
- **File picker** — click the blue *browse* link to open the operating system's standard file dialog.

Files up to 50 MB are accepted. The `.log`, `.txt`, and `.out` extensions are recognised, but the tool reads the file by content, so any plain-text log will work.

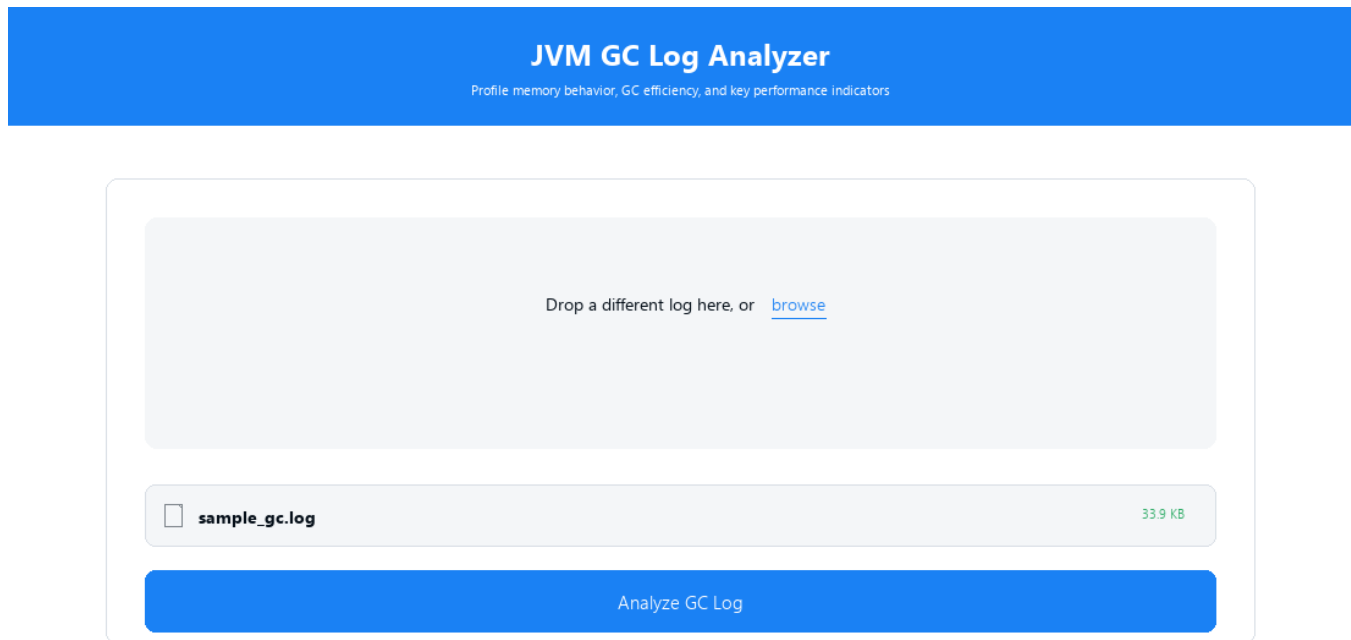


Figure 2 — After a file is selected, a confirmation pill appears below the drop area showing the filename and size, and the 'Analyze GC Log' button becomes active.

3.3 — Running the analysis

Click **Analyze GC Log**. The application uploads the file to the local server, parses every GC event, computes statistics, generates charts, and runs the observation engine. A spinner with the message “*Parsing events and computing metrics...*” is displayed during processing. For typical log sizes this takes a few seconds.

If the file cannot be parsed (for example because it is not a valid GC log), a red-bordered error message explains the problem and the analysis does not start.

3.4 — Reviewing the results

The result page is organised top-down so the most important information is visible first:

- **File badge** — a small chip near the top confirms which file is being analysed.
- **Overview tiles** — six headline metrics: events parsed, log duration, total pause time, application throughput, GC rate, and total memory reclaimed.
- **GC Event Breakdown** and **Pause Duration** — a two-column view showing what kinds of GC ran and how long each pause was.
- **Heap Metrics** — heap usage before and after GC, plotted over time with chart-level observations underneath.
- **GC Duration over time** and **Reclaimed Memory over time** — additional time-series visualisations.
- **Key Performance Indicators (KPI) Dashboard** — a unified view of throughput, CPU time, latency percentiles, pause distribution, and duration buckets.

- **Observations** — severity-tagged findings (OK / INFO / WARN) drawn from the entire analysis, with plain-language explanations.
- **Action buttons** — Download PDF Report and Analyze Another File.

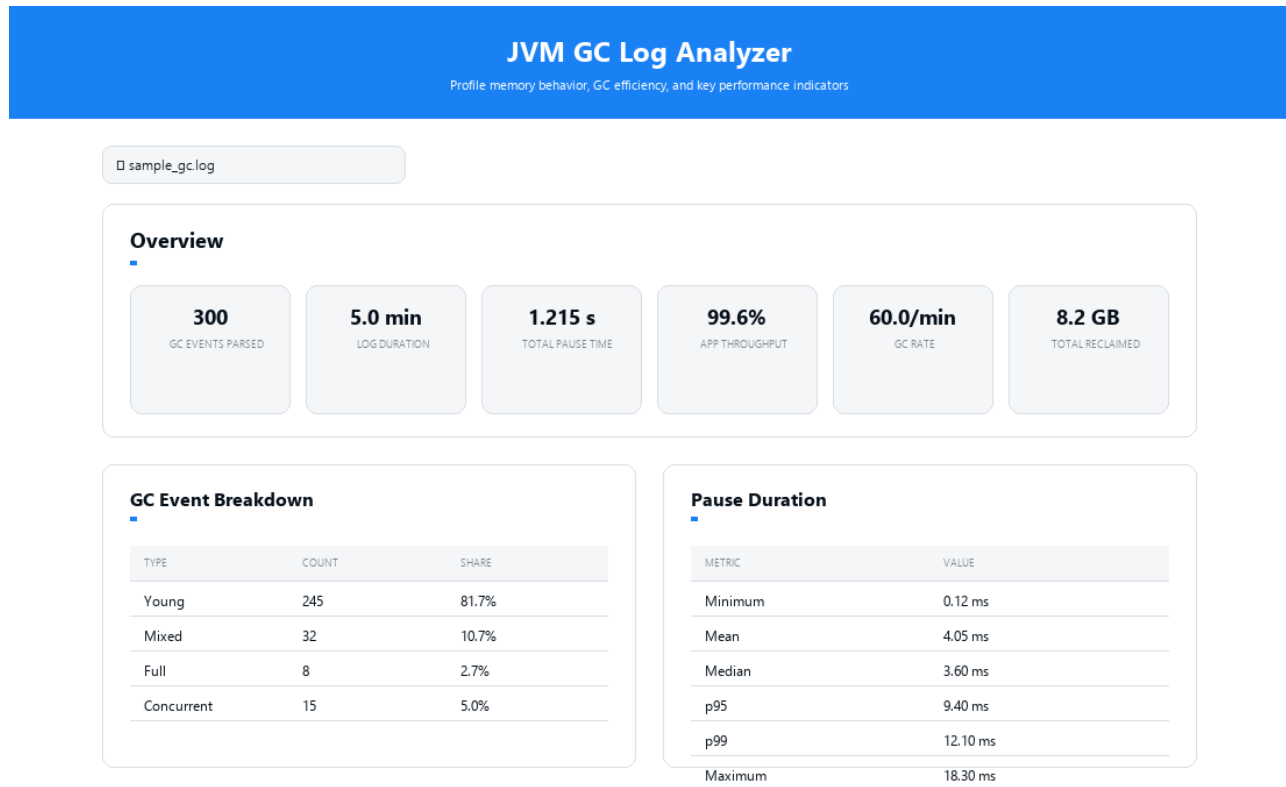


Figure 3 — The top of the results page: a file badge confirms what was analysed, followed by six headline KPI tiles, the GC-event breakdown, and the pause-duration percentiles.

3.5 — Interpreting graphs, metrics, and observations

Each part of the result page has a specific purpose. The notes below describe what to look for and what each finding usually means.

Heap charts

The *heap-before-GC* chart shows how much memory was in use just before each garbage collection event. A healthy line shows a roughly steady level or a gentle sawtooth (allocate, then reclaim). A line that climbs continuously across the full timeline often indicates a memory leak or growing retained state — the observation engine flags this explicitly. The *heap-after-GC* chart shows how much memory remained after each reclaim; rising lows are the strongest leak signal.

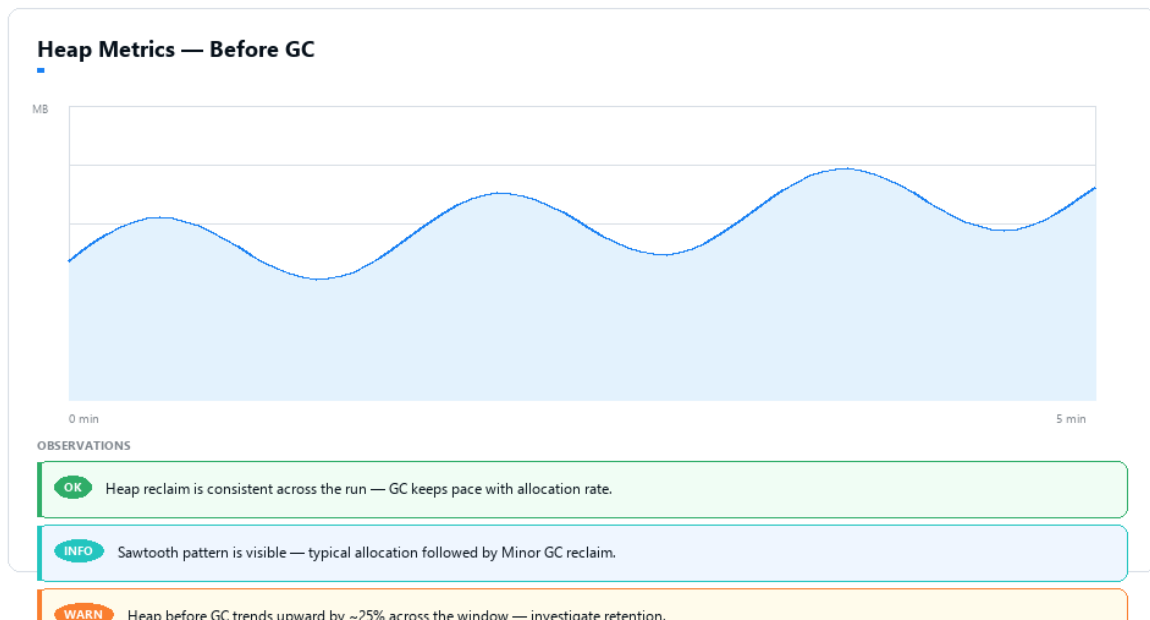


Figure 4 — A heap chart with the per-chart observations rendered immediately beneath it. Severity colours match the rest of the UI: green for OK, teal for INFO, orange for WARN.

Pause-duration distribution and KPIs

Throughput is the most important single number: it is the percentage of wall-clock time the JVM spent *not* in GC. A value above 95% is normal for most applications; values below 90% suggest GC is taking a significant share of CPU. Pause-time percentiles (p95, p99) tell you about the user-visible impact: if your service has a 50 ms latency budget, a p99 GC pause of 100 ms means one in a hundred requests will be slowed by GC alone.



Figure 5 — The KPI dashboard. Headline tiles surface the most important measurements; the bar chart underneath shows how pauses are distributed across duration buckets.

Observation severity

Observations are written by the tool's rule engine and are not subjective — every finding describes a measurable property of the log. Three severity levels are used:

[OK]	Confirms a metric is in a healthy range. No action required.
[INFO]	Provides context worth knowing but not necessarily acting on. Useful for understanding the workload.
[WARN]	Flags a measurement that is out of range or trending in a concerning direction. Investigate.

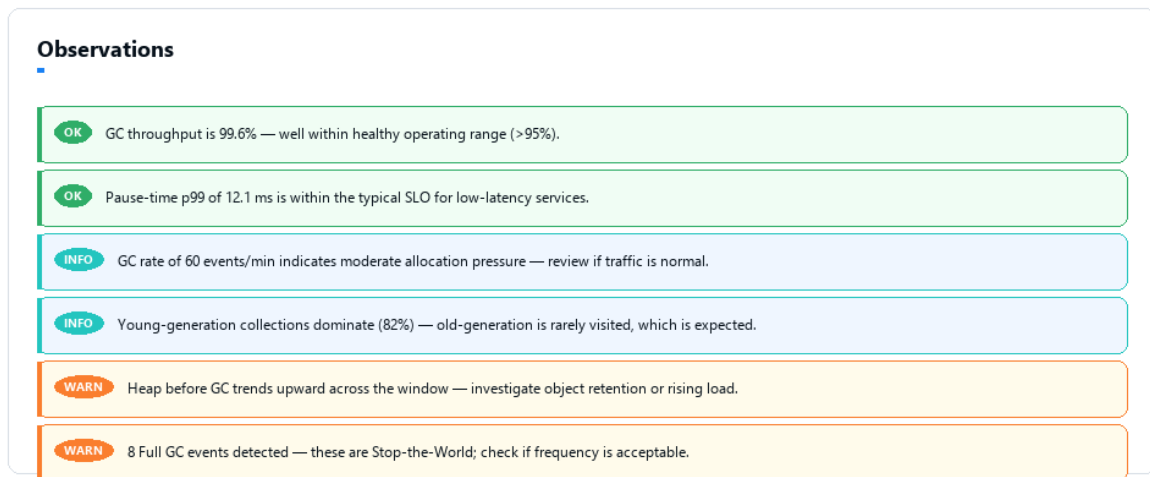


Figure 6 — A sample observations list. Severity is shown both as a coloured tag and as a left-rule on the card, so it is visible at a glance.

3.6 — Downloading the analysis report as a PDF

At the bottom of the results page, click **Download PDF Report**. The application renders a multi-page PDF containing the file metadata, all KPIs, every chart, the full observation list, and (where applicable) the G1GC-specific timing breakdowns. The PDF downloads immediately and is ready to attach to a ticket, send in chat, or store in a shared drive.

Throughput and Heap Memory image analyses (described in section 3.8) each have their own PDF export. Those PDFs use a timestamped filename format — for example `2026-05-20_1430_Throughput_Image_Analysis.pdf` — so successive exports never overwrite earlier ones.

3.7 — Using the Analyze Another File button

Below the PDF download button, the secondary **Analyze Another File** button clears the current results and returns to the upload screen. The previous file is forgotten and no data is retained — use this when you want to compare a series of logs (for example, before and after a JVM tuning change).

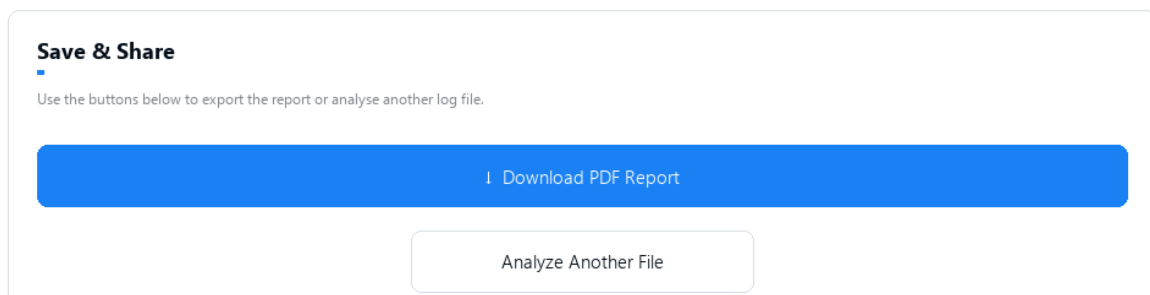


Figure 7 — The two action buttons at the bottom of the results page. The primary download button is filled blue; the secondary 'Analyze Another File' button is white-on-light-gray to make the action hierarchy clear.

3.8 — Throughput and Heap Memory image analysis (optional)

Below the GC log upload card, the page also exposes two image-analysis features designed for screenshots exported from the Management Console:

- **Throughput Image Analysis** — expects `Throughput_30days.png`, `Throughput_7days.png`, and `Throughput_24hours.png` in a folder you specify.
- **Heap Memory Image Analysis** — expects `HeapMemory_30days.png`, `HeapMemory_7days.png`, and `HeapMemory_24hours.png`.

For each analyser, paste the folder path containing the screenshots and click the **Analyze ...** button. The tool extracts the graph line from each image, classifies the trend (Stable, Increasing, Decreasing, Fluctuating, or Showing sudden drops or spikes), and produces a cross-window comparison that highlights short-term issues invisible in the longer-term graphs.

Heap memory analysis specifically watches for the classic memory-leak signature (rising mean across 30-day → 7-day → 24-hour) and for stable-but-high heap usage that indicates memory pressure with limited headroom.

Throughput Image Analysis

Provide the folder containing the throughput screenshots exported from the Management Console. The analyzer will look for `Throughput_30days.png`, `Throughput_7days.png` and `Throughput_24hours.png`, identify each graph, and produce a detailed cross-window review of throughput behaviour.

Analyze Throughput Images

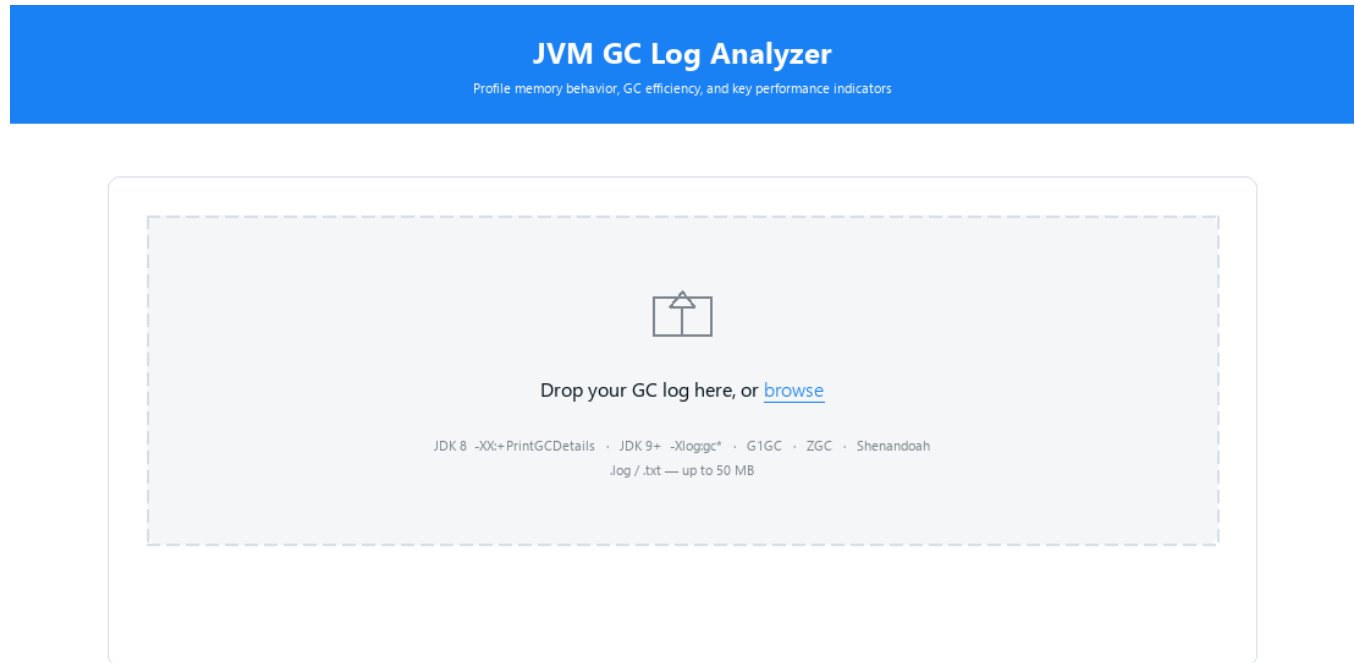
WHEN ANALYSIS COMPLETES ...

Each image is identified, classified (Stable / Increasing / Decreasing / Fluctuating / Sudden drops or spikes), and an actionable cross-window comparison is generated. A PDF export with a timestamped filename is available right below the results.

Figure 8 — The Throughput Image Analysis card. The Heap Memory equivalent sits directly below it and behaves identically, with heap-specific interpretation.

4. Screenshots Gallery

The illustrations below are stylised representations of the application's interface. They use the same colour tokens, typography, and layout as the live application so they remain accurate even as the underlying screenshots evolve.



Upload screen — empty state ready for a GC log.

JVM GC Log Analyzer

Profile memory behavior, GC efficiency, and key performance indicators

Drop a different log here, or [browse](#)

sample_gc.log

33.9 KB

Analyze GC Log

File selected — confirmation pill and analyse button enabled.

JVM GC Log Analyzer

Profile memory behavior, GC efficiency, and key performance indicators

sample_gc.log

Overview

300

GC EVENTS PARSED

5.0 min

LOG DURATION

1.215 s

TOTAL PAUSE TIME

99.6%

APP THROUGHPUT

60.0/min

GC RATE

8.2 GB

TOTAL RECLAIMED

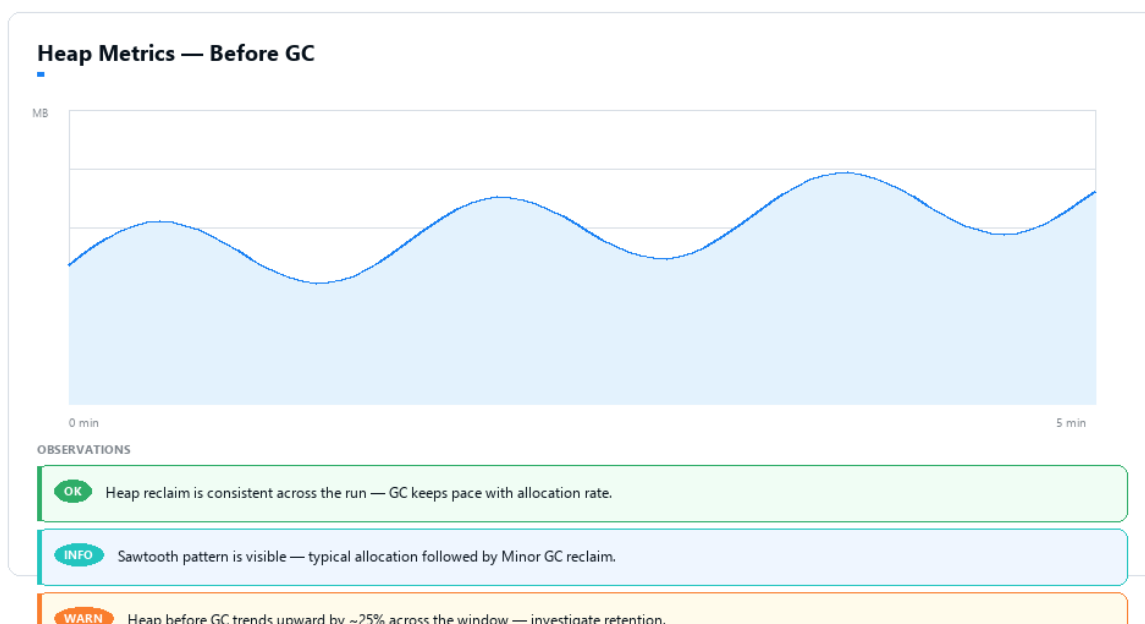
GC Event Breakdown

TYPE	COUNT	SHARE
Young	245	81.7%
Mixed	32	10.7%
Full	8	2.7%
Concurrent	15	5.0%

Pause Duration

METRIC	VALUE
Minimum	0.12 ms
Mean	4.05 ms
Median	3.60 ms
p95	9.40 ms
p99	12.10 ms
Maximum	18.30 ms

Results — overview KPIs, GC-event breakdown, pause percentiles.



Heap chart with per-chart observations.



KPI dashboard with pause-time distribution.

Observations

- OK** GC throughput is 99.6% — well within healthy operating range (>95%).
- OK** Pause-time p99 of 12.1 ms is within the typical SLO for low-latency services.
- INFO** GC rate of 60 events/min indicates moderate allocation pressure — review if traffic is normal.
- INFO** Young-generation collections dominate (82%) — old-generation is rarely visited, which is expected.
- WARN** Heap before GC trends upward across the window — investigate object retention or rising load.
- WARN** 8 Full GC events detected — these are Stop-the-World; check if frequency is acceptable.

Severity-tagged observation cards (OK / INFO / WARN).

Save & Share

Use the buttons below to export the report or analyse another log file.

1 Download PDF Report

Analyze Another File

Action buttons — Download PDF and Analyze Another File.

Throughput Image Analysis

Provide the folder containing the throughput screenshots exported from the Management Console.
The analyzer will look for Throughput_30days.png, Throughput_7days.png and Throughput_24hours.png,
identify each graph, and produce a detailed cross-window review of throughput behaviour.

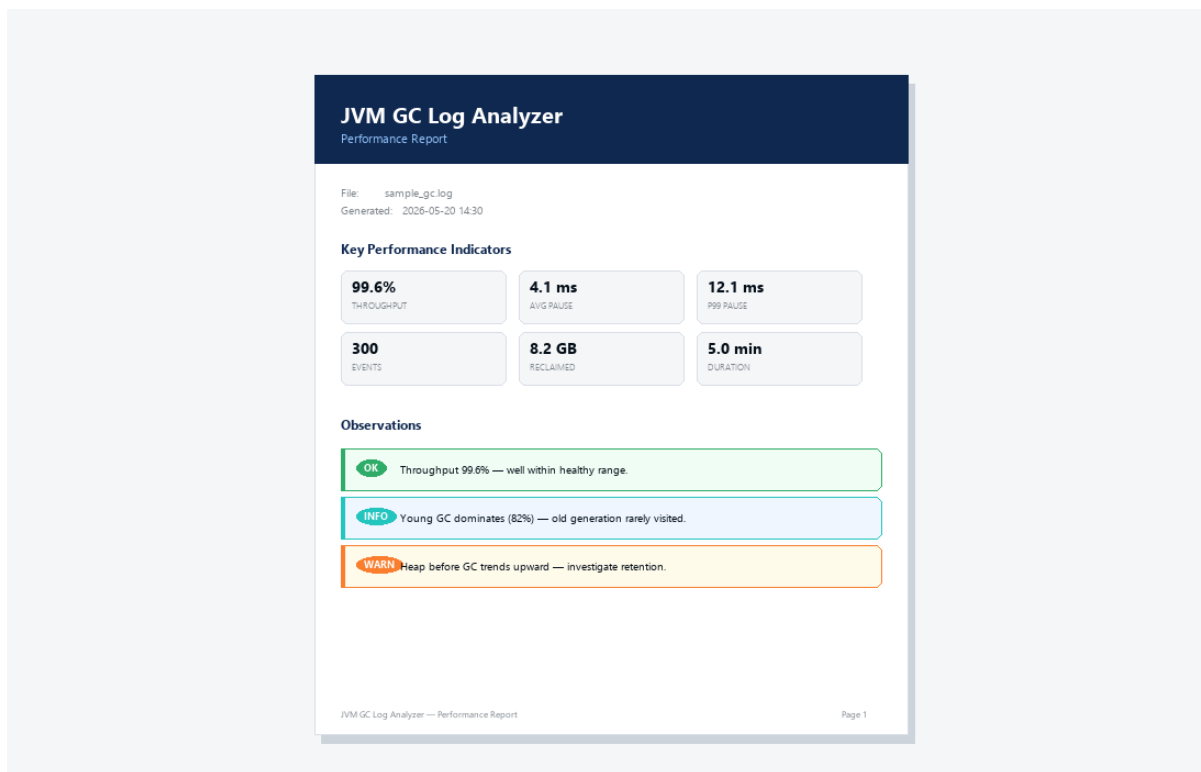
C:\exports\mgmt-console\

Analyze Throughput Images

WHEN ANALYSIS COMPLETES ...

Each image is identified, classified (Stable / Increasing / Decreasing / Fluctuating / Sudden drops or spikes),
and an actionable cross-window comparison is generated. A PDF export with a timestamped
filename is available right below the results.

Throughput / Heap Memory image-analysis card.



Preview of the downloadable PDF report.

5. Output Report

The Download PDF Report button produces a printable, share-ready summary of the analysis. It is rendered server-side, embeds the same charts shown on screen, and uses the same design tokens as the web interface for visual consistency. The sections below describe what to expect inside.

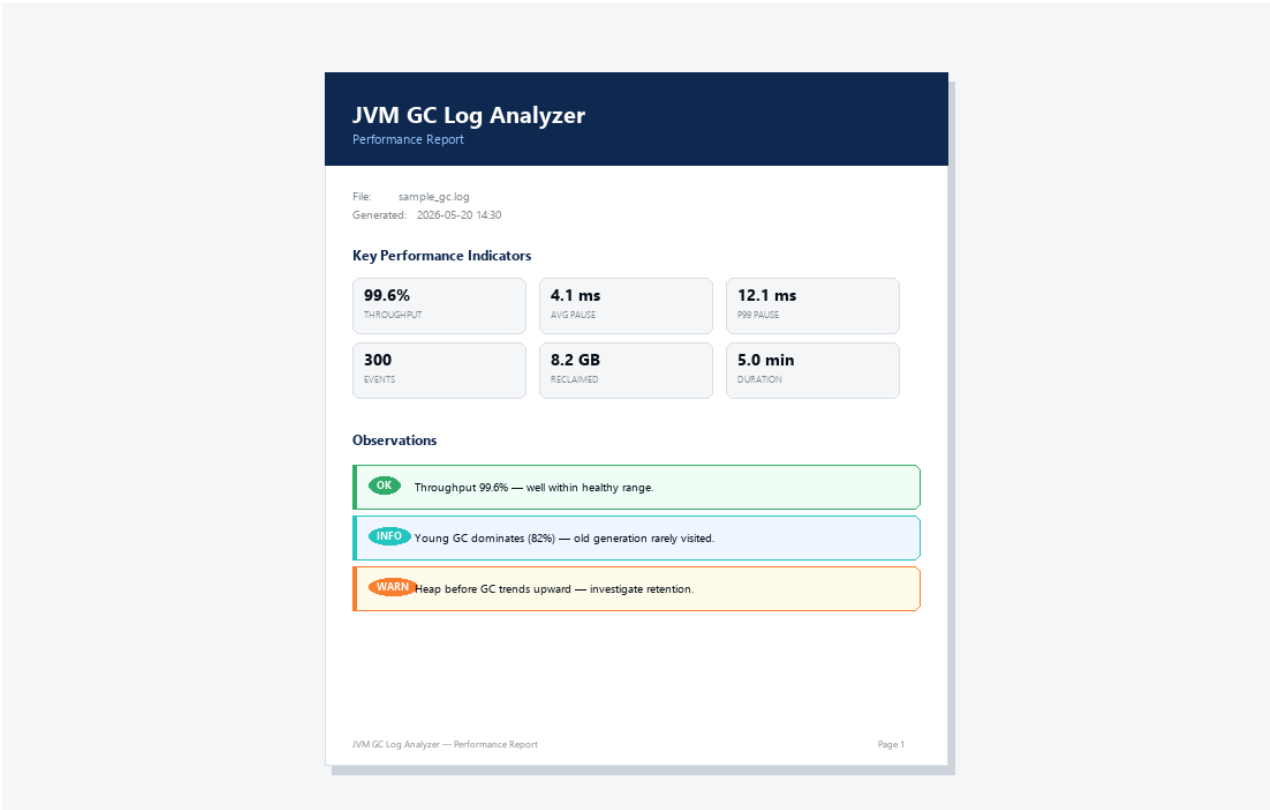


Figure 9 — Preview of the downloadable PDF report.

Sections inside the PDF report

Section	What it contains
Header banner	JVM GC Log Analyzer title and 'Performance Report' subtitle.
File metadata	Original log filename and generation timestamp.
Key Performance Indicators	Throughput, average pause, p95 / p99 pauses, total pause time, GC rate, total reclaimed.
GC Event Breakdown	Counts and percentages by GC type (Young, Mixed, Full, Concurrent, Unknown).
Pause Duration	Min / mean / median / p95 / p99 / max pause statistics.
Heap Metrics	Heap-before and heap-after maxima, minima, and reclaimed totals.
Heap charts	Heap-before-GC and heap-after-GC plotted over time.
GC Duration chart	Pause duration plotted over time, colour-coded by GC type.

Reclaimed Memory chart	Memory reclaimed per event, plotted over time.
KPI Dashboard	Summary tiles plus pause-duration bucket distribution.
Observations	Severity-tagged findings from the entire analysis.
G1GC details (if applicable)	Pause-time and concurrent-phase breakdowns when a G1-format log is detected.
Footer	Page numbers and report identifier on every page.

Reading the observations

Observations are the most actionable part of the report. They are produced by deterministic rules — not subjective judgements — so two people running the same log will see the same findings. The three severity levels are read consistently across the application: green is healthy, teal is contextual, orange asks for attention.

Health assessment summary

A run can be considered **healthy** when throughput exceeds 95%, p99 pause-time is within the application's latency budget, the heap-before-GC line is flat or sawtoothed (not climbing), Full GC frequency is low, and no WARN-level observations are produced. Any of these failing is reason to investigate further — typically by tuning the heap size, switching collectors, or addressing object-retention issues in the application code.

Technical findings

For deeper investigation, the report includes the raw inputs the observations are built from: full statistical tables, bucket distributions, and time-series traces. These are intended for performance engineers who want to verify the conclusions or extract the data for further analysis in other tools.

6. Supported Formats and Limits

Item	Detail
GC log size limit	50 MB per upload.
GC log file types	.log, .txt, .out (any plain-text format).
JDK versions	JDK 8 and JDK 9 through current.
Garbage collectors	Serial, Parallel, CMS (legacy), G1GC, ZGC, Shenandoah.
Image analysis	PNG files exported from the Management Console.
Expected image names	Throughput_{30days 7days 24hours}.png, HeapMemory_{30days 7days 24hours}.png
Network use	None — the application runs entirely on localhost.
Persistent storage	None — uploaded files are processed in-memory and discarded.

For project setup, source code, and contributor notes, see the [README](#) and [CLAUDE.md](#) files in the project root.